Mobile Application Development Assignment

App Name: DarkLogin

Student Name: Yunxuan Peng

ID: 20223803032

Date: 2025.11.23

1. Basic Design & Implementation

1.1 Introduction

This application, **"DarkLogin,"** is a secure authentication app developed using **Kotlin** and **Jetpack Compose**. This means that all code can be written in a single .kt file, and to meet and exceed the assignment requirements, I designed a custom "Deep Space" theme featuring animated backgrounds and glassmorphism effects. The application architecture is based on a single-activity model with state-driven navigation.

Below is the detailed implementation of each interface and feature.

1.2 Login Interface

The Login Screen is the entry point of the application. It features a **glassmorphism card** that floats over the animated background.

Key Features & Logic:

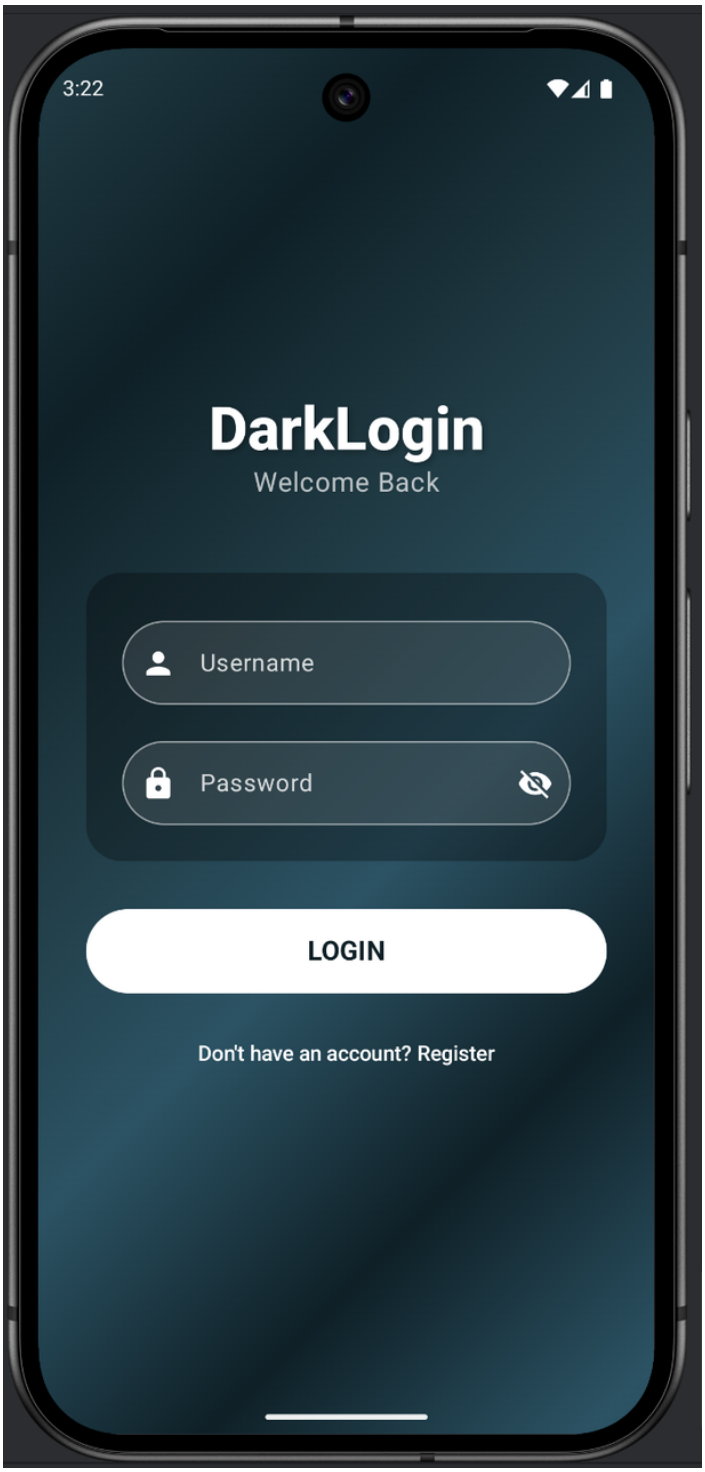
1. **First Run Check:** When the user clicks "LOGIN" for the very first time (before any account is created), the app detects that the local database is empty and shows a Toast message.
2. **Navigation to Registration:** Users can easily navigate to the registration screen by clicking the "Register" button if they don't have an account.
3. **Authentication:** It validates the input against stored credentials.

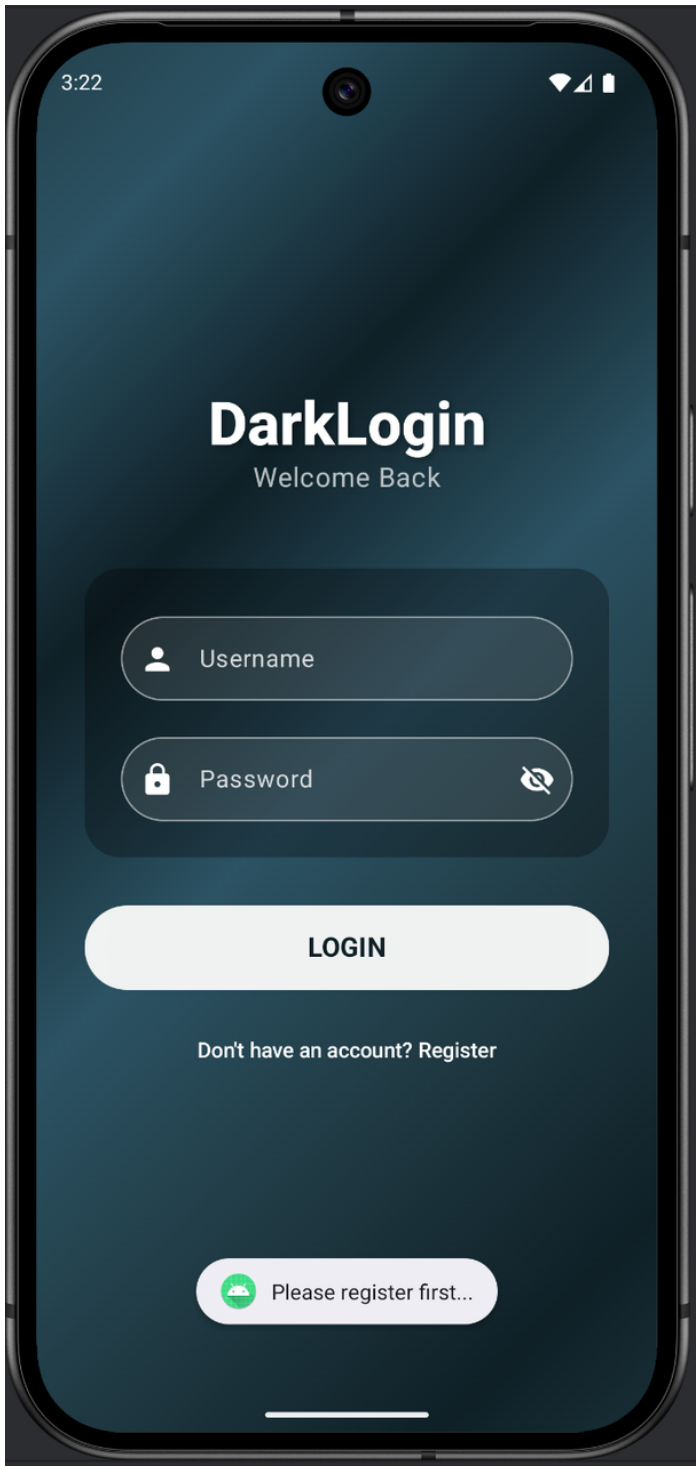
Code Snippet (First Login Logic):

代码块

```
// 1. Check if no users are registered yet
if (userDatabase.isEmpty()) {
    Toast.makeText(context, "Please register first...",
Toast.LENGTH_SHORT).show()
    return@LoginScreen false
}

// 2. Navigation callback when Register button is clicked
TextButton(onClick = onGoToRegister) {
    Text("Don't have an account? Register", color = Color.White)
}
```





1.3 Registration Interface

Users navigate to this screen by clicking "Register" on the login page. The UI maintains visual consistency with the login screen but focuses on account creation.

Key Features & Logic:

1. **Data Storage:** Since no external database was required, I used a `MutableMap` to store usernames and passwords in memory.
2. **Password Visibility:** To improve user experience and reduce typing errors, I implemented a **visibility toggle (eye icon)** in the password field. This allows users to verify their password before registering.

3. **Auto-Navigation:** Upon successful registration, the app automatically navigates back to the Login screen so the user can sign in.

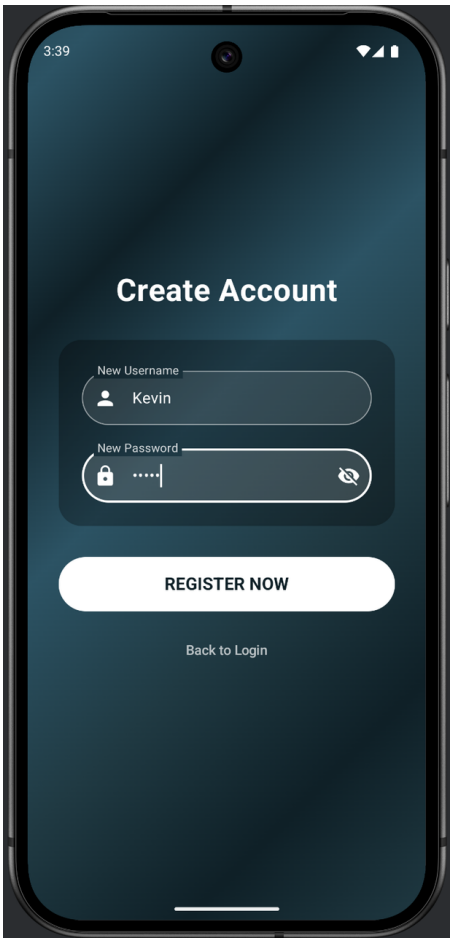
Code Snippet:

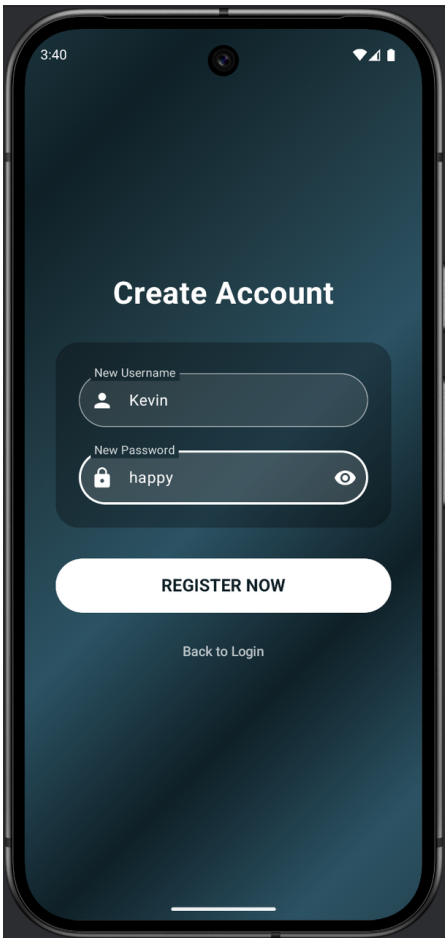
代码块

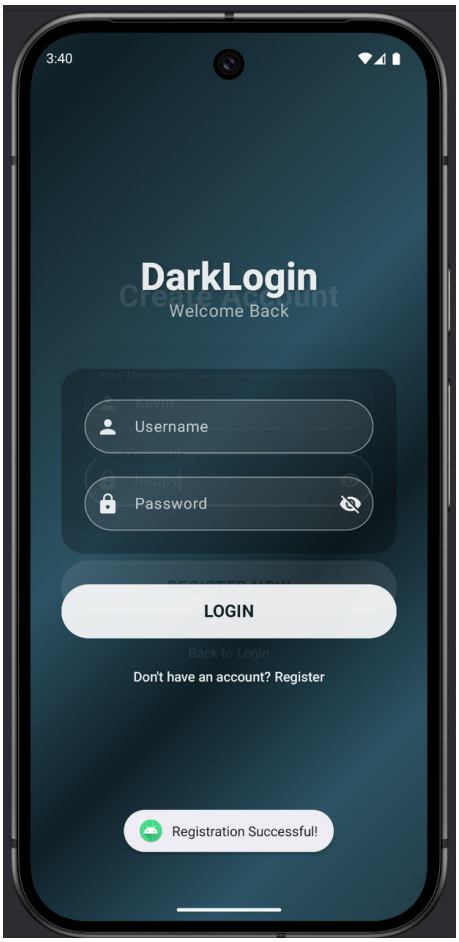
```
// 1. Data Storage: In-memory map defined in parent scope
val userDatabase = rememberSaveable { mutableMapOf<String, String>() }

// 2. Auto-Navigation: Logic executed when 'Register' is clicked
onRegister = { username, password ->
    userDatabase[username] = password // Save credentials
    currentScreen = Screen.LOGIN      // Navigate back automatically
}

// 3. Password Visibility: Toggle icon logic in TextField
trailingIcon = if (isPassword) {
    {
        IconButton(onClick = { passwordVisible = !passwordVisible }) {
            Icon(
                if (passwordVisible) Icons.Default.Visibility else
Icons.Default.VisibilityOff,
                contentDescription = "Toggle Visibility",
                tint = Color.White
            )
        }
    }
} else null
```







1.4 Error Display & Lockout Mechanism

This is a critical security feature required by the assignment.

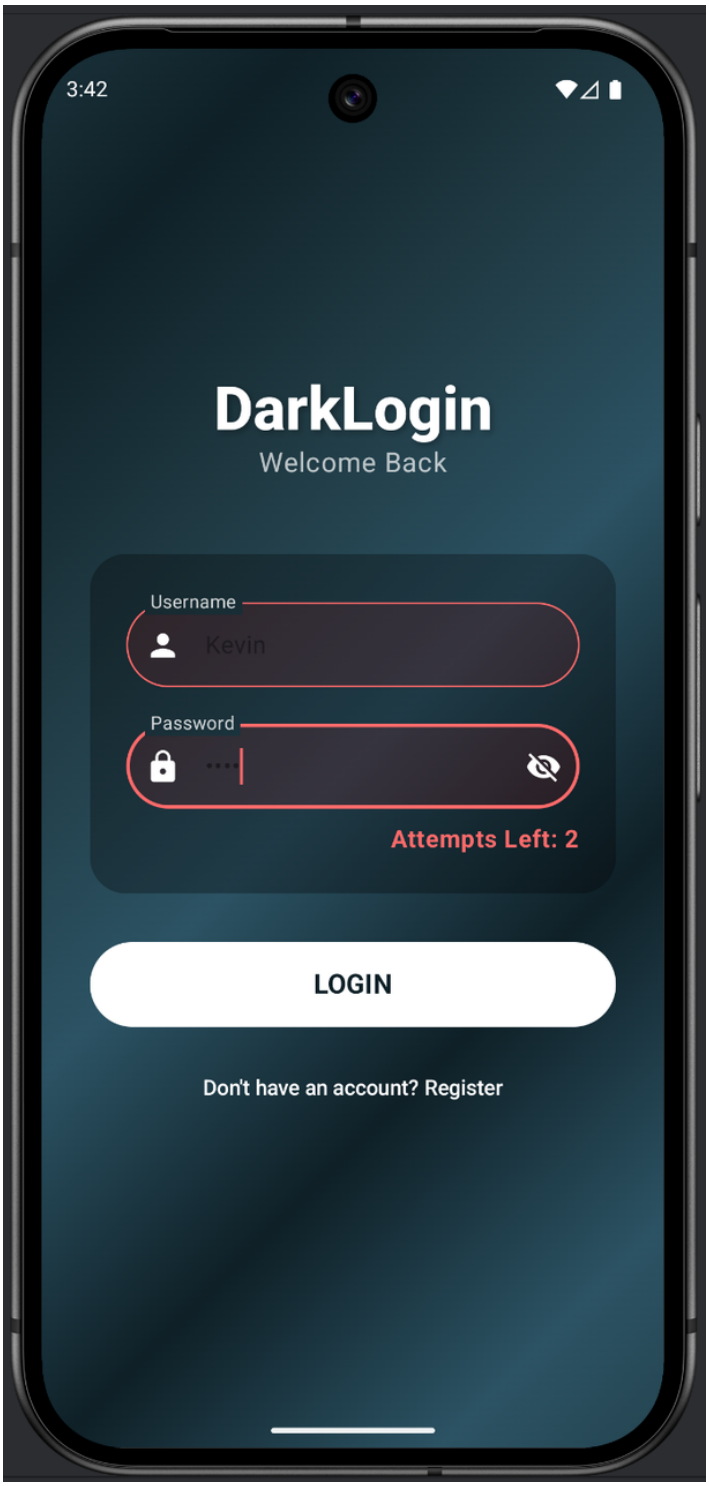
Key Features & Logic:

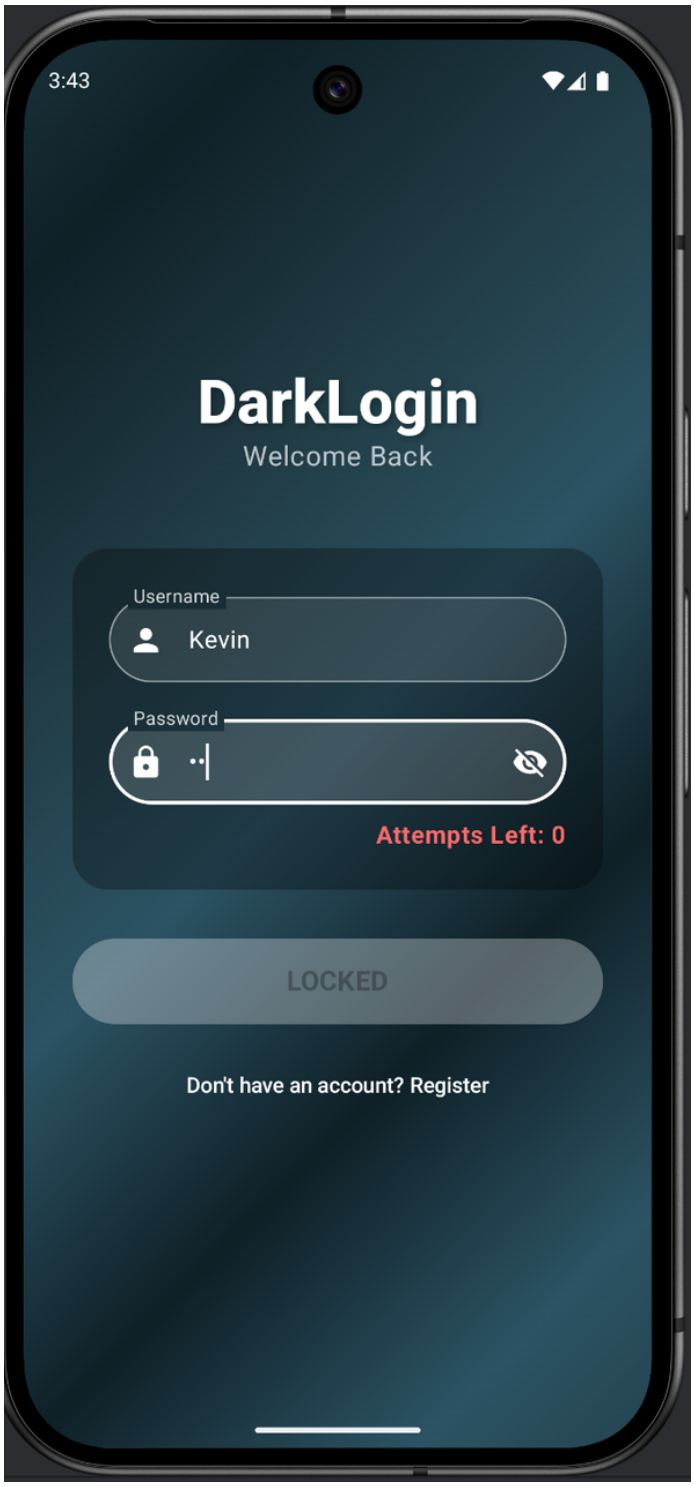
1. **Visual Feedback:** If the password is wrong, the input fields are highlighted (or a Toast appears), and the "Attempts Left" counter decreases.
2. **Lockout Logic:** A state variable `attemptsLeft` initializes at 3. When it reaches 0, the Login button is visibly disabled (turned gray) to prevent brute-force attacks.

Code Snippet (Button Disabling):

代码块

```
// The button's 'enabled' state is bound to the counter
Button(
    onClick = { /* Login Logic */ },
    enabled = attemptsLeft > 0, // Button becomes unclickable when 0
    colors = ButtonDefaults.buttonColors(
        containerColor = if (attemptsLeft > 0) Color.White else Color.Gray
    )
) {
    Text(if (attemptsLeft > 0) "LOGIN" else "LOCKED")
}
```





1.5 Success Screen & Logout

Once authentication is successful, the user is taken to the Welcome Screen.

Key Features & Logic:

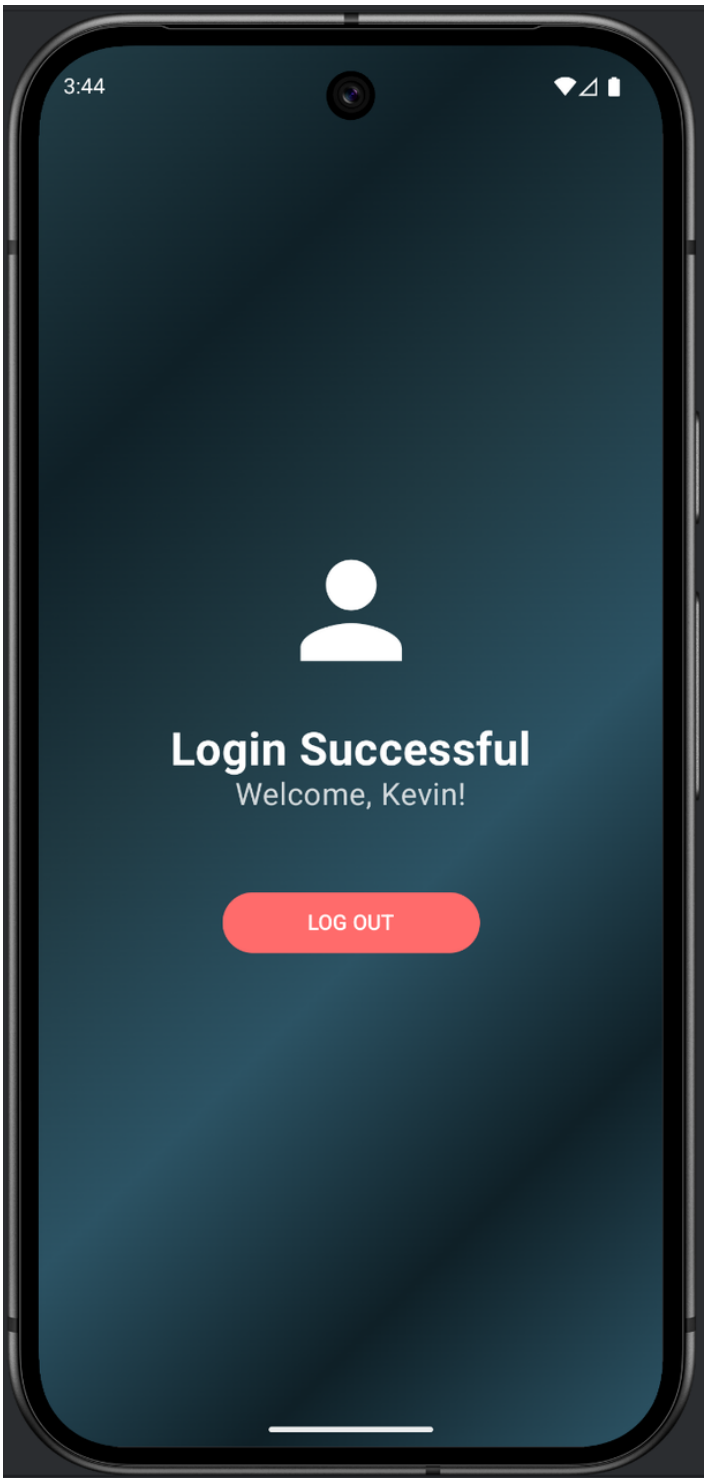
1. **Dynamic Welcome:** The screen displays a personalized message: `"Welcome + username!"`.
2. **Logout Functionality:** A "LOGOUT" button is provided to reset the session and return the user to the Login screen.

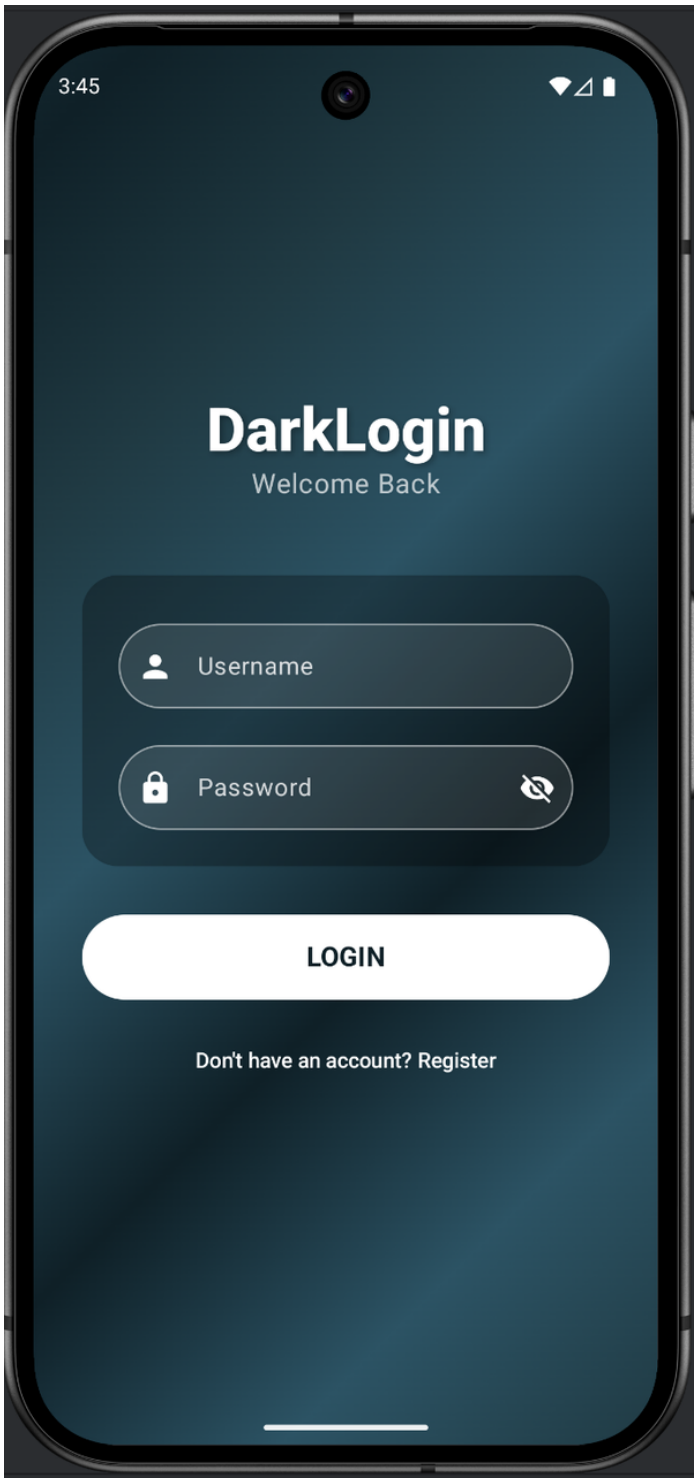
Code Snippet (Navigation & Logout Logic):

代码块

```
// 1. Login Success: Verify credentials and navigate to Success Screen
if (userDatabase[username] == password) {
    activeUser = username
    currentScreen = Screen.SUCCESS
    true
}

// 2. Logout Logic: Button click resets state to Login Screen
Button(
    onClick = { currentScreen = Screen.LOGIN }, // Logic passed to onLogout
    shape = RoundedCornerShape(50),
    colors = ButtonDefaults.buttonColors(containerColor = Color(0xFFFF6B6B))
) {
    Text("LOG OUT", color = Color.White)
}
```





2. Problems, Solutions, and Future Improvements

2.1 Problems and Solutions

During the development process, I encountered several challenges:

1. **Problem: Visual Consistency.**
    - *Issue:* Standard Android buttons looked out of place on the dark animated background.
    - *Solution:* I created custom Composable functions with specific `Color` and `Shape` parameters to ensure all buttons and inputs shared the same "DarkLogin" aesthetic.
2. **Problem: State Loss on Rotation.**

- *Issue:* The `attemptsLeft` counter would reset to 3 if the user rotated the phone.
- *Solution:* I used `rememberSaveable` instead of `remember`. This allows the attempt counter to survive configuration changes.

2.2 Shortcomings

- **Data Persistence:** The user data is currently stored in memory (`mutableMapOf`). If the app is closed completely, data is lost.
- **Security:** Passwords are stored in plain text memory.

2.3 Future Improvements

- **Database Integration:** Implement **Room Database** to permanently save user credentials.
- **Biometric Auth:** Add Fingerprint unlock features.